

Esame OS Internals (Cabodi)

Data: 27/06/2020

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Contesto: Si consideri il seguente frammento di programma:

```
#define N 128
#define M 64
int i, j, k;
float *A[N], B[N][M];

for(i=0; i<N; i++) {
    A[i] = calloc(M, sizeof(float));
    for (j=0; j<M; j++) {
        B[i][j] = i * j;
    }
}

for(k=0; k<N*M; k++) {
    i = rand() % N;
    j = rand() % M;
    A[i][j] += B[i][j];
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 4 Kbyte, utilizzando come politica di sostituzione pagine la LRU (Least Recently Used). Si sa che i puntatori occupano 64 bit, mentre sia `int` che `float` occupano 32 bit.

Si risponda alle seguenti domande:

-) Si calcoli la dimensione massima possibile (in Byte e in numero di pagine/frame) del Virtual Address Space (VAS) di un processo, e la dimensione massima della RAM.
-) Si calcoli la dimensione in memoria (espressa in numero di pagine) del codice e dei dati, necessari all'esecuzione del frammento proposto.
-) Supponendo che il resident set per il processo sia fisso a F frames. Supponendo di disporre dei 3 frame necessari per codice, stack e array `A`, quanti frame supplementari occorre assegnare affinché il numero di page fault sia minimizzato per la matrice `B`?

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF), il tempo medio per riparazione (MTTR), e il tempo medio tra perdite di dato (MTTDL).

Si risponda alle seguenti domande:

-) A causa di vincoli aggiuntivi, la probabilità di guasto di un disco, mentre l'altro è in riparazione, viene RADDOPPIATA. Si consideri l'effetto sull'MTTDL.
 -) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTF = 20000$ ore, quanto deve essere MTTR per garantire $MTTDL > 100 \cdot MTTF$?
-

Domanda 3 (I/O, Polling e Interrupt)

Si risponda alle seguenti domande:

-) Si spieghino le principali differenze tra I/O sincrono e asincrono. Si considerino poi I/O bloccante e non bloccante: si tratta di sinonimi di asincrono e sincrono?
-) Un I/O sincrono può essere effettuato in polling oppure è necessario effettuarlo in interrupt? (Rispondere alla stessa domanda per il caso dell'I/O asincrono).
-) In quale modo può un processo beneficiare di un I/O asincrono? E' possibile, nel caso di I/O asincrono, scrivere un programma con istruzioni, successive all'I/O, che dipendano dai dati letti durante l'I/O?

SECTION 2: OS161

Domanda 4 (Configurazione Kernel)

Contesto: Sia dato un sistema OS161. Si supponga di aver aggiunto a `kern/conf/conf.kern` le istruzioni:

```
defoption exam
optfile exam syscall/exam.c
```

e di aver creato in `kern/conf` il file `EXAM`, copiato dal file `DUMBVM`.

Si risponda alle seguenti domande:

-) Si dica se sono sufficienti le azioni descritte, più l'esecuzione in `kern/conf` di `./config EXAM`, affinché venga compilato il file opzionale `syscall/exam.c`.
 -) Quale, tra i file `exam.h` e `opt-exam.h`, viene generato automaticamente dal comando `./config EXAM`? Viene generato sempre, oppure solo se in `EXAM` compare l'istruzione `options exam`?
 -) Cosa contiene il file generato automaticamente?
 -) Si supponga di inserire, nel file `main.c`, l'istruzione `exam_init()`; . Come è possibile fare in modo che l'istruzione sia presa in considerazione e compilata solamente nelle versioni del kernel in cui viene abilitata l'opzione `exam`?
-

Domanda 5 (Processi e I/O)

Contesto: Si consideri un processo utente su OS161.

Si risponda alle seguenti domande:

-) Spiegare brevemente la differenza tra `switchframe` e `trapframe`. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la `thread_fork`?
-) Perché i dispositivi di I/O sono mappati su `kseg1`? Sarebbe possibile mappare un dispositivo di I/O in `kseg0`?
-) Un processo utente può eseguire una `write(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`?

ANSWERS

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Dati Iniziali:

- $N = 128, M = 64$.
- Dimensione pagina = 4 KB = 4096 Byte.
- Puntatori = 64 bit (8 Byte).
- `int` = 32 bit (4 Byte), `float` = 32 bit (4 Byte).

A) Dimensione massima del VAS e della RAM:

Il sistema utilizza puntatori a 64 bit. Questo significa che l'architettura è in grado di indirizzare 2^{64} locazioni di memoria distinte (byte).

- **VAS (in Byte):** 2^{64} Byte.
- **VAS (in Pagine):** $\frac{2^{64}}{2^{12}} = 2^{52}$ pagine.
- **RAM massima:** Considerando un'architettura standard in cui il bus degli indirizzi fisici coincide con quello logico, la RAM massima è anch'essa di 2^{64} Byte (ovvero 2^{52} frame fisici).

B) Dimensione in memoria (Pagine) di codice e dati:

Analizziamo lo spazio occupato dalle strutture dati:

- Array di puntatori $A[N]$: 128×8 Byte = 1024 Byte. Interamente contenuto in **1 pagina**.
- Matrice $B[N][M]$: $128 \times 64 \times 4$ Byte = 32.768 Byte. Occupa $32768/4096 = 8$ pagine.
- Memoria heap allocata per $A[i]$: Il ciclo esegue 128 chiamate a `calloc`, ognuna da $64 \times 4 = 256$ Byte. Totale = $128 \times 256 = 32.768$ Byte. Occupa **8 pagine**.
- Le variabili intere (`i`, `j`, `k`) occupano 12 Byte (tipicamente allocate nei registri o in **1 pagina** di Stack).
- Il segmento di codice occupa tipicamente **1 pagina**.

C) Frame supplementari per la matrice B:

Nel secondo ciclo, gli accessi alle matrici `A` e `B` avvengono in modo pseudo-casuale (`rand()`). Se la matrice `B` non fosse interamente presente in RAM, ogni accesso casuale rischierebbe di generare un Page Fault, penalizzando gravemente le prestazioni (*thrashing*).

Poiché la matrice `B` occupa esattamente **8 pagine** logiche (32.768 Byte), per "minimizzare" i page fault azzerando i *capacity/conflict misses* durante la fase di accesso randomico, occorre assegnare **esattamente 8 frame supplementari** dedicati interamente a mantenere `B` in memoria nel Resident Set.

Domanda 2 (RAID e Affidabilità)

A) Effetto sull'MTTDL se la probabilità di guasto raddoppia:

L'MTTDL di un sistema RAID tollerante ai guasti è inversamente proporzionale alla probabilità che un guasto critico avvenga durante la finestra di riparazione ($P_{fatal} = C \cdot \frac{MTTR}{MTTF}$). Se per vincoli aggiuntivi tale probabilità raddoppia ($2 \cdot P_{fatal}$), il denominatore della formula dell'MTTDL viene moltiplicato per 2. Di conseguenza, il **MTTDL viene esattamente DIMEZZATO**.

B) Calcolo MTTR per RAID 1:

In un RAID 1 a due dischi (Mirroring), la formula dell'affidabilità è $MTTDL = \frac{MTTF^2}{2 \cdot MTTR}$. Imponiamo il vincolo richiesto dalla traccia:

$$\frac{MTTF^2}{2 \cdot MTTR} > 100 \cdot MTTF \implies \frac{MTTF}{2 \cdot MTTR} > 100 \implies MTTR < \frac{MTTF}{200}$$

Sostituendo il valore noto ($MTTF = 20.000$ ore):

$$MTTR < \frac{20.000}{200} \implies \mathbf{MTTR < 100 \text{ ore}}$$

—

Domanda 3 (I/O Polling e Interrupt)

A) Differenze I/O e Sinonimi:

- **Sincrono vs Asincrono:** Nell'I/O sincrono, la system call ritorna il controllo al chiamante solo dopo il completamento fisico del trasferimento. Nell'I/O asincrono, la chiamata ritorna istantaneamente avviando l'operazione in background.
- **Sinonimi? NO.** Bloccante/Non-Bloccante sono sottocategorie dello *scheduling* POSIX per l'I/O sincrono. L'I/O non-bloccante ritorna subito un errore (es. **EAGAIN**) se i dati non sono pronti, ma **non avvia** alcun trasferimento in background (a differenza dell'asincrono).

B) Necessità degli Interrupt:

- L'I/O **Sincrono** può essere effettuato in **entrambi** i modi: tramite Polling (busy-waiting della CPU) o tramite Interrupt (sleep/wakeup).
- L'I/O **Asincrono** necessita **obbligatoriamente degli Interrupt**. Poiché la CPU non è bloccata e sta eseguendo il codice utente in parallelo all'I/O hardware, l'unico modo per il dispositivo di segnalare il completamento dell'operazione in background è sollevare un'eccezione hardware (Interrupt).

C) Benefici dell'I/O Asincrono e Dipendenze Dati:

Il beneficio primario è la **sovrapposizione (overlap)** tra le fasi di esecuzione della CPU e le fasi di accesso ai dispositivi (I/O burst), permettendo di massimizzare l'efficienza.

SÌ, è possibile eseguire istruzioni dipendenti dai dati asincroni, ma tali istruzioni **non possono seguire sequenzialmente** la system call iniziale. Devono essere implementate all'interno di un *Signal Handler* o di una funzione di *Callback* che il kernel invocherà autonomamente non appena l'Interrupt notificherà la disponibilità dei dati.

—

Domanda 4 (Configurazione Kernel)

A) Sufficienza delle azioni in `conf.kern`:

NO, non sono sufficienti.

Le direttive in `conf.kern` si limitano a dichiarare l'opzione al sistema di configurazione. Per renderla effettiva e ordinare la compilazione di `exam.c`, occorre aprire il file di configurazione `kern/conf/EXAM` e aggiungervi esplicitamente la riga: `options exam`.

B & C) File generato e suo contenuto:

Il comando `./config EXAM` genera **sempre** il file `opt-exam.h`, a prescindere dal fatto che l'opzione sia stata abilitata o meno nel file `EXAM`.

Il file contiene esclusivamente la definizione di una singola macro C:

- `#define OPT_EXAM 1` (se `options exam` era presente).
- `#define OPT_EXAM 0` (se l'opzione non era dichiarata).

D) Compilazione condizionale in `main.c`:

Per inserire il blocco in modo sicuro si utilizza il costrutto `#if` (non `#ifdef`, poiché la macro esiste sempre). Occorre anche includere l'header autogenerato.

```
#include "opt-exam.h"
```

```
#if OPT_EXAM
    exam_init();
#endif
```

—

Domanda 5 (Processi e I/O)

A) Switchframe vs Trapframe e relativo utilizzo:

- **Trapframe:** Salva **tutto** lo stato della CPU (tutti i registri). Si innesca tramite hardware durante le eccezioni per passare da User a Kernel Mode.
- **Switchframe:** Salva **solo** i registri *callee-save*. Viene generato via software per i context switch volontari tra kernel threads.

Casi d'uso:

- Avviare un processo utente $\implies$ **Trapframe**.
- Gestire una chiamata di sistema $\implies$ **Trapframe**.
- Operazione di `thread_fork` $\implies$ **Switchframe**.

B) Mappatura dei dispositivi I/O (`kseg1` vs `kseg0`):

I dispositivi (Memory-Mapped I/O) sono mappati nel `kseg1` perché è l'unico segmento hardware della CPU **non soggetto a caching** (uncached).

Non è possibile mapparli in `kseg0` perché essendo un segmento controllato dalla Cache, le operazioni di scrittura verso il dispositivo hardware rimarrebbero intrappolate nella Cache L1, e le operazioni di lettura restituirebbero dati stantii ignorando i cambiamenti hardware in tempo reale.

C) System Call con buffer in `kseg0`:

NO, non è assolutamente permesso.

Il segmento `kseg0` è memoria riservata al Kernel. Il processo risiede nello spazio non privilegiato (`kuseg`). Quando viene invocata la system call, le funzioni interne del Kernel preposte alla copia (es. `copyin`) verificheranno la provenienza dell'indirizzo e, rilevando l'accesso a memoria kernel, faranno fallire la chiamata restituendo un errore (`EFAULT`) per proteggere l'integrità del sistema operativo.